

CHAPTER 1: INTRODUCTION

1.1 Introduction

Artificial Intelligence (AI) has transformed how systems solve problems and assist decision-making. Traditional software works on predefined rules, which limits its ability to handle complex and dynamic situations.

To overcome this, Multi-Agent Systems (MAS) have been introduced, where multiple intelligent agents collaborate to achieve a common goal. These systems provide better scalability, flexibility, and efficiency.

With advancements in Large Language Models (LLMs), modern AI systems can understand natural language, perform reasoning, and generate solutions. This has led to the development of autonomous multi-agent systems capable of handling complex workflows.

The project “**Autonomous Cognitive Machine (Cognix)**” is a multi-agent system that automates problem-solving through task planning, execution, validation, and synthesis using technologies like LangGraph, LangChain, Mistral AI, and Streamlit.

1.2 Background of the Study

AI has evolved from rule-based systems to advanced models capable of learning and reasoning. Machine Learning and Deep Learning improved adaptability and performance in various domains.

Modern LLMs such as GPT, Claude, and Mistral can generate text, code, and insights. However, single-model systems face challenges like limited scalability and lack of specialization.

To solve this, Agentic AI systems use multiple specialized agents working collaboratively. Cognix follows this approach to efficiently handle complex tasks using a structured workflow.

1.3 Need for the Project

Modern systems face challenges such as:

- Complex task execution
- Information overload
- Lack of intelligent automation
- Scalability issues

Cognix addresses these by introducing a multi-agent system capable of autonomous decision-making and execution.

1.4 Objectives

- Task planning using AI
 - Autonomous execution
 - Agent collaboration
 - Output validation
 - Report generation
 - User-friendly interface
-

1.5 Scope

The system supports:

- Research automation
 - Code generation
 - Task execution
 - Report generation
 - Knowledge summarization
-

1.6 Chapter Summary

This chapter introduced the concept of Autonomous Cognitive Multi-Agent Systems and discussed the motivation behind the development of the Cognix platform. The chapter highlighted the need for intelligent collaborative agents, defined the project objectives, identified the problem statement, and provided an overview of the proposed solution. The subsequent chapters will discuss the organization profile, literature survey, system architecture, implementation details, testing procedures, and project outcomes in greater detail.

CHAPTER 2 : LITERATURE SURVEY

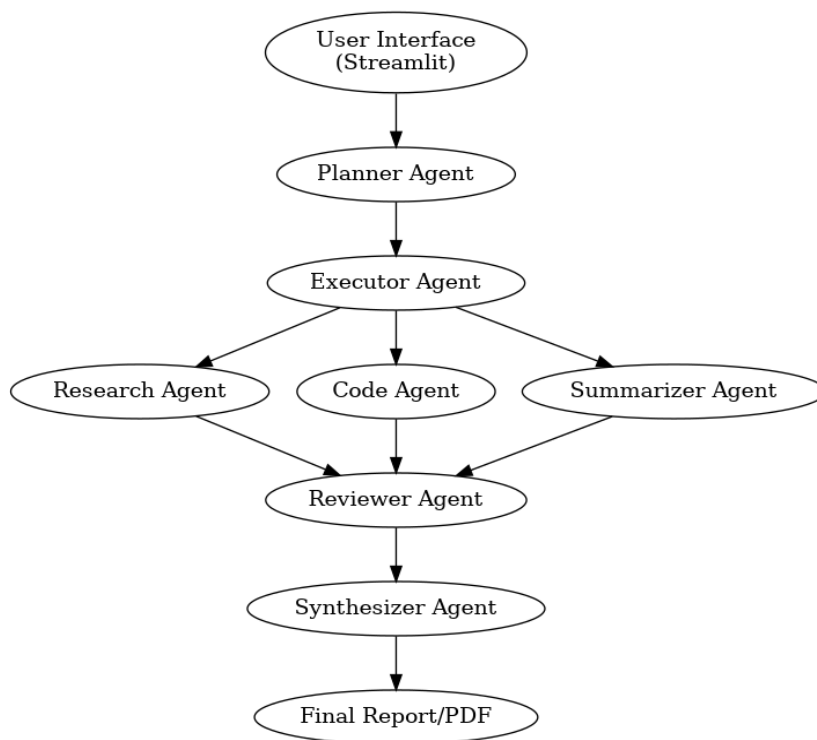
2.1 Overview

Modern AI systems combine Multi-Agent Systems, Cognitive Computing, and LLMs to enable intelligent automation.

2.2 Multi-Agent Systems

- Multiple agents collaborate
- Distributed execution
- Improves scalability

Figure : Multi-Agent System Architecture Diagram



2.3 Cognitive Computing

- Mimics human reasoning
 - Supports decision-making
 - Adapts dynamically
-

2.4 Large Language Models

- Perform reasoning, coding, summarization
 - Used as intelligence engine
-

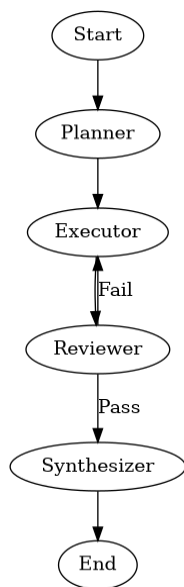
2.5 Agentic AI

- Uses Planner, Executor, Reviewer
 - Enables autonomous workflows
-

2.6 LangChain & LangGraph

- LangChain → LLM integration
- LangGraph → workflow orchestration

Figure : LangGraph Workflow Diagram



2.7 LangSmith

- Monitoring and debugging tool
 - Tracks execution flow
-

2.8 Chapter Summary

This chapter discussed the key technologies and concepts underlying the Cognix system, including Multi-Agent Systems, Cognitive Computing, Large Language Models, and Agentic AI. It highlighted how multiple intelligent agents collaborate to perform complex tasks efficiently. The chapter also covered important frameworks such as LangChain for model integration, LangGraph for workflow orchestration, and LangSmith for monitoring and debugging. Additionally, it emphasized the role of Mistral AI models and knowledge retrieval systems in enhancing system performance. Overall, the concepts presented in this chapter provide the foundation for designing a scalable, autonomous, and intelligent multi-agent system.

CHAPTER 3 : REQUIREMENT ANALYSIS

3.1 Problem Statement

Traditional systems struggle with:

- Multi-step task execution
- Lack of automation
- Poor scalability

3.2 Proposed Solution

Cognix uses:

- Planner
- Executor
- Agents
- Reviewer
- Synthesizer

3.3 Functional Requirements

- Goal input
- Task generation
- Execution
- Validation
- Report generation

3.4 Non-Functional Requirements

- Performance
 - Scalability
 - Reliability
 - Security
-

Figure : Requirement Analysis Diagram

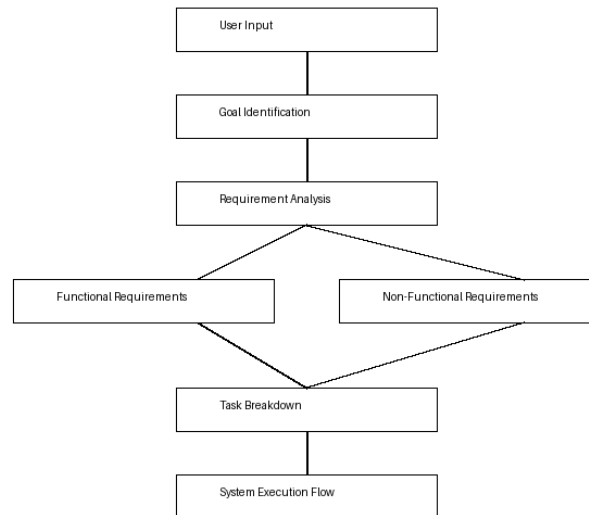
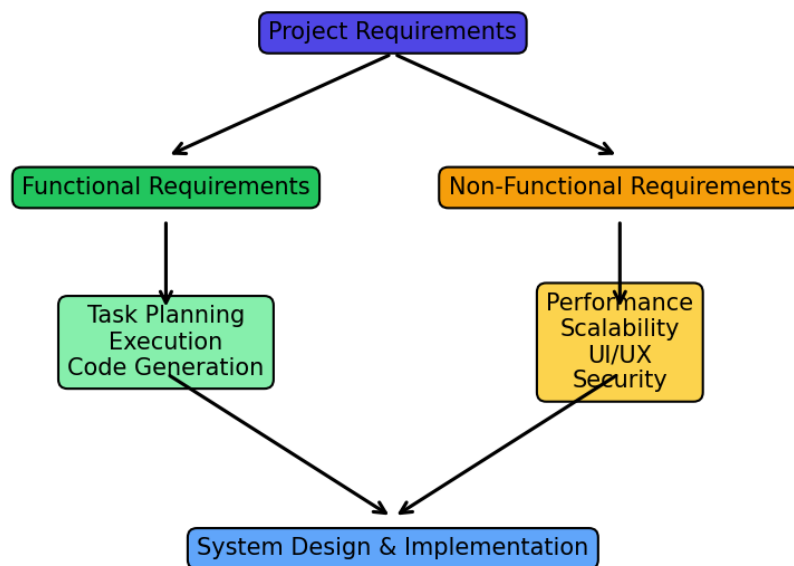


Figure : Functional and Non-Functional Requirements Diagram



3.5 Chapter Summary

This chapter analyzed the problems in traditional AI systems and defined the requirements for the proposed Cognix system. It presented the problem statement, functional and non-functional requirements, and compared existing systems with the proposed multi-agent architecture. The analysis demonstrated the need for a scalable, autonomous system capable of task planning, execution, and validation.

CHAPTER 4 : SYSTEM ARCHITECTURE

4.1 Overview

The Cognix system follows a structured workflow where user goals are processed through multiple stages. Each stage is handled by a dedicated agent responsible for a specific task.

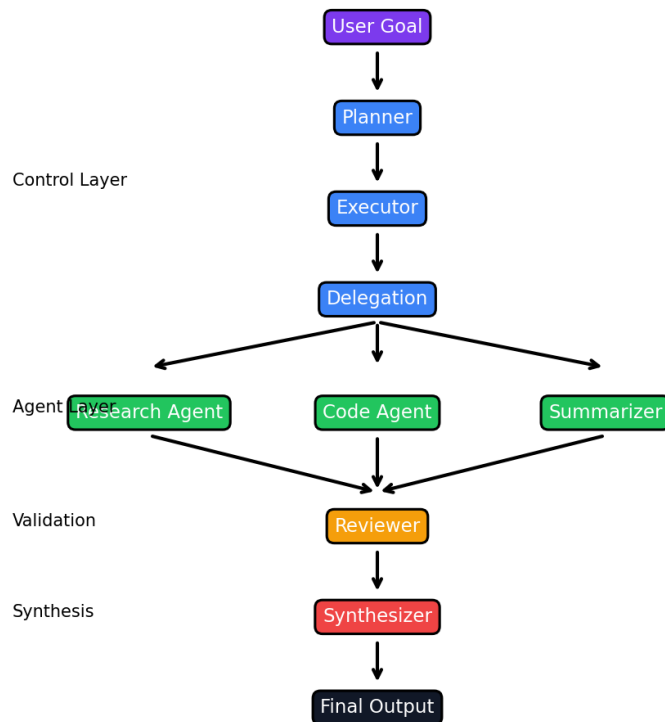
4.2 Components

- User Interface (Streamlit)
 - Planner Agent
 - Executor Agent
 - Research Agent
 - Code Agent
 - Summarizer Agent
 - Reviewer Agent
 - Synthesizer Agent
-

4.3 Workflow

1. User enters goal
 2. Planner generates tasks
 3. Executor processes tasks
 4. Agents perform execution
 5. Reviewer validates
 6. Synthesizer creates report
-

4.4 Overall Cognix System Architecture



4.5 Chapter Summary

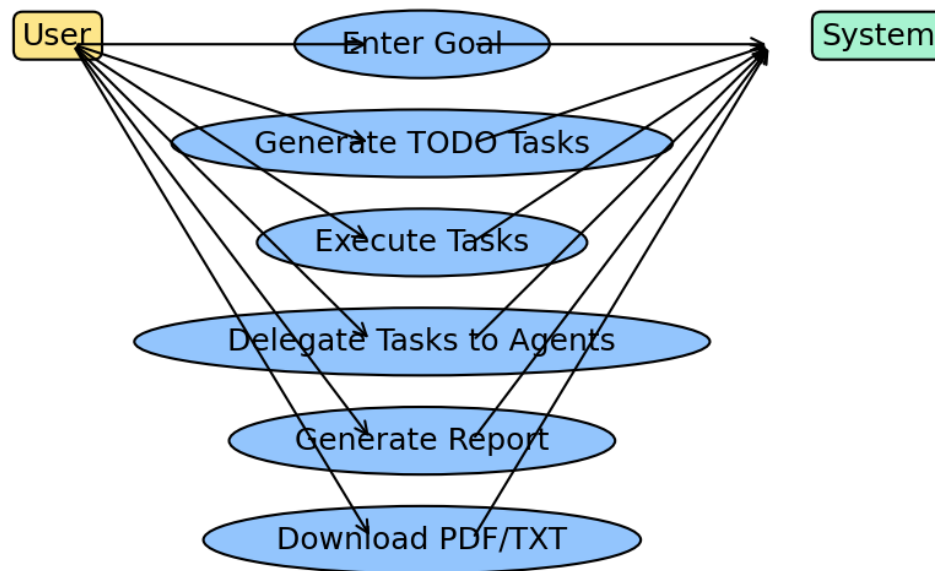
This chapter explained the overall architecture of the Cognix system, describing its multi-agent design and workflow. It outlined the roles of key components such as the Planner, Executor, Research, Code, Summarizer, Reviewer, and Synthesizer agents. The chapter showed how these components interact to process user goals, execute tasks, validate outputs, and generate final reports.

CHAPTER 5 : SYSTEM DESIGN (UML)

5.1 Use Case Diagram

A Use Case Diagram describes the interaction between the user and the Cognix system. It identifies the major functionalities provided by the system and how users interact with them.

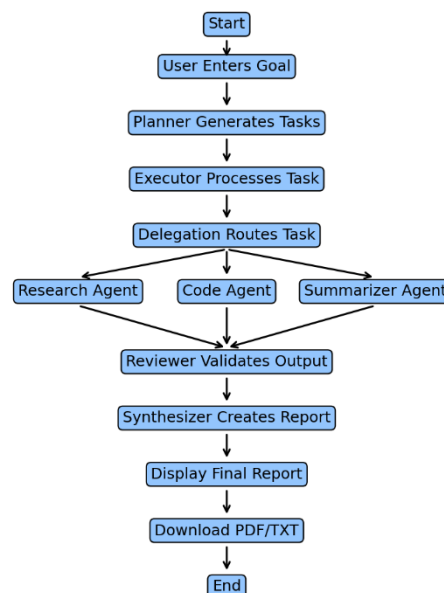
Figure:



5.2 Activity Diagram

The Activity Diagram illustrates the workflow of the Cognix system from goal submission to final report generation.

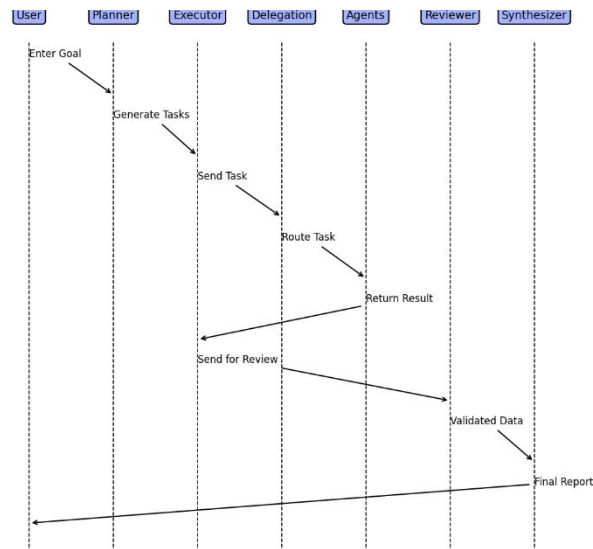
Figure:



5.3 Sequence Diagram

A Sequence Diagram represents the interaction among different components in chronological order.

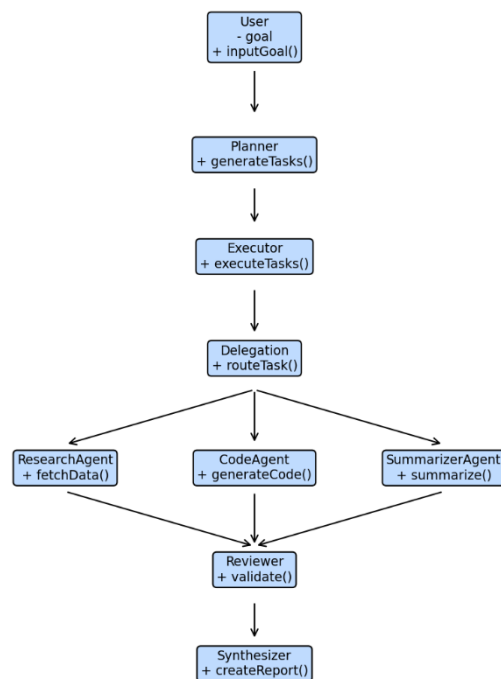
Figure:



5.4 Class Diagram

The Class Diagram provides a static representation of the system structure.

Figure:



5.5 Chapter Summary

This chapter presented the design of the Cognix system using UML diagrams such as Use Case, Activity, Sequence, and Class diagrams. These diagrams provided a visual representation of system interactions, workflow execution, and structural relationships among components, helping to better understand the behavior and design of the system.

CHAPTER 6 : SYSTEM IMPLEMENTATION

6.1 Introduction

System implementation is the phase where the designed architecture is converted into a working software application. The Autonomous Cognitive Machine (Cognix) was developed using Python and modern AI frameworks such as LangGraph, LangChain, Mistral AI, Streamlit, Tavily Search API, and LangSmith.

The implementation focuses on creating a collaborative multi-agent environment capable of planning, executing, reviewing, and synthesizing tasks autonomously. The system provides an interactive user interface and supports automated report generation.

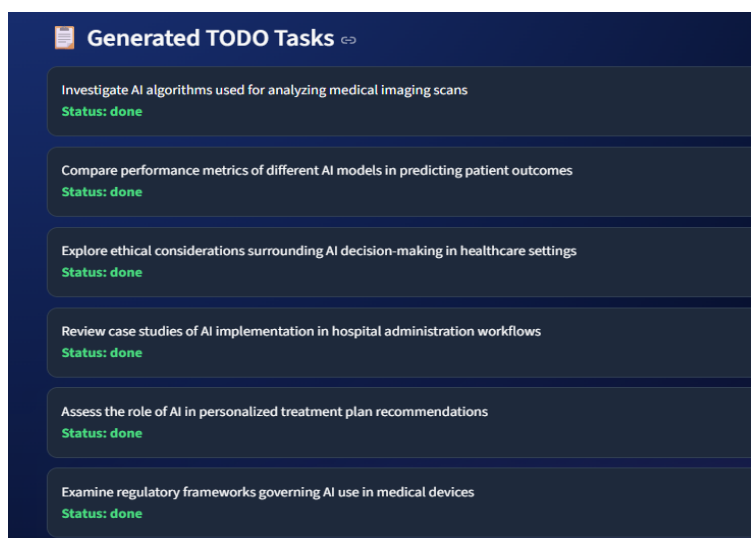
6.2 Technologies Used

- Python
 - LangGraph
 - LangChain
 - Streamlit
 - Mistral AI
 - ReportLab
-

6.3 Module Implementation

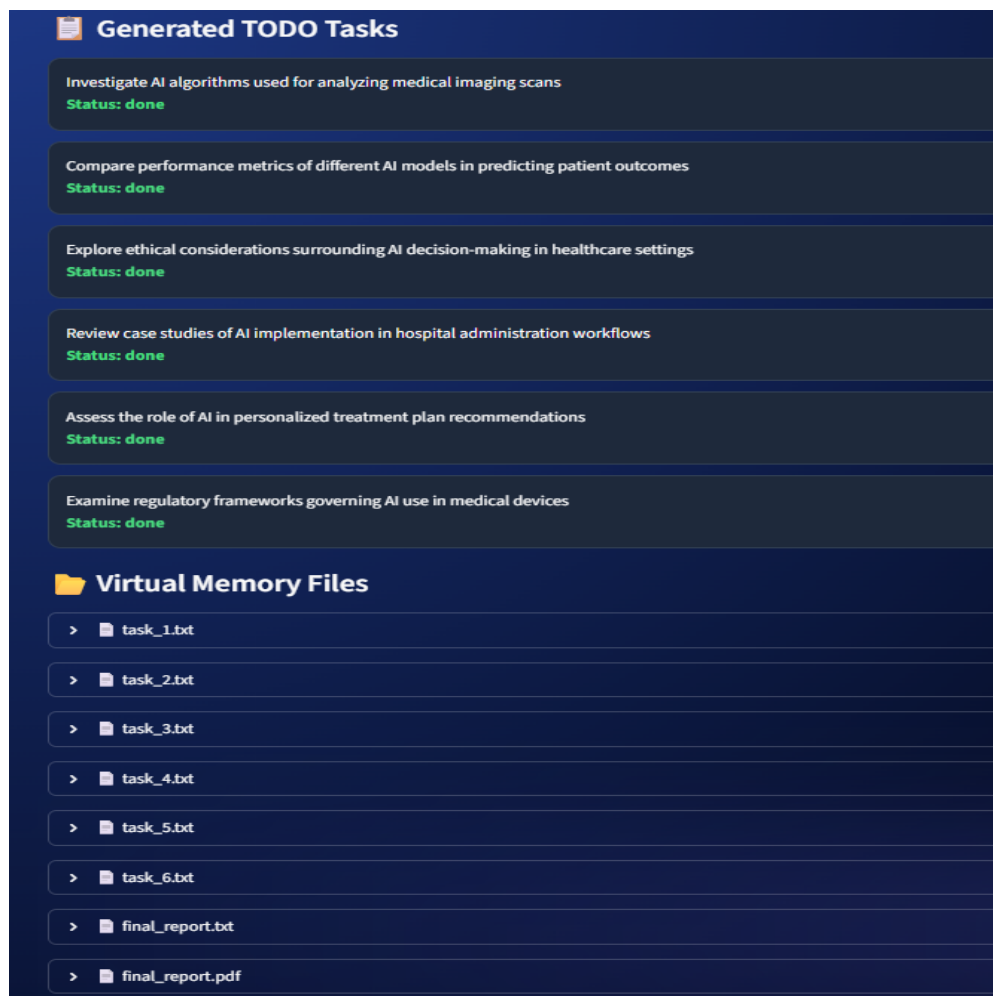
Planner Agent

- The Planner Agent is the first cognitive component of the system.



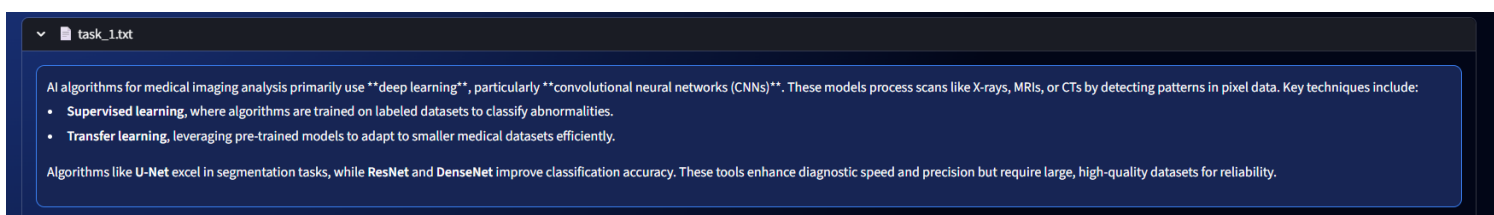
Executor Agent

- The Executor Agent is responsible for carrying out the tasks generated by the Planner Agent.



Research Agent

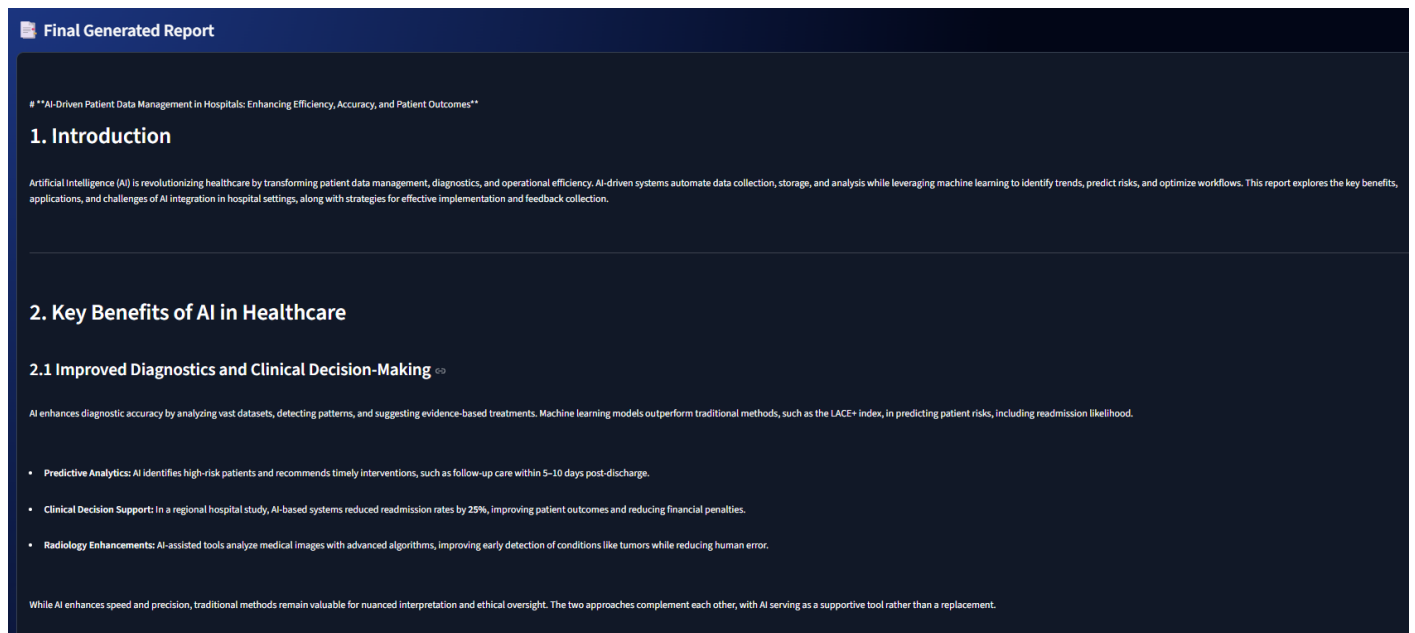
- The Research Agent gathers information from external sources using the Tavily Search API.



Code Agent

- The Code Agent specializes in programming and technical tasks.

- The Synthesizer Agent combines outputs from all agents into a single structured response



PDF and TXT Report Generation Module

- The PDF and TXT Report Generation Module is responsible for creating the final output of the Cognix system. It organizes the processed results into a structured format and exports them as PDF and text files, ensuring professional, readable, and easily shareable reports.

Figure : Generated PDF Report

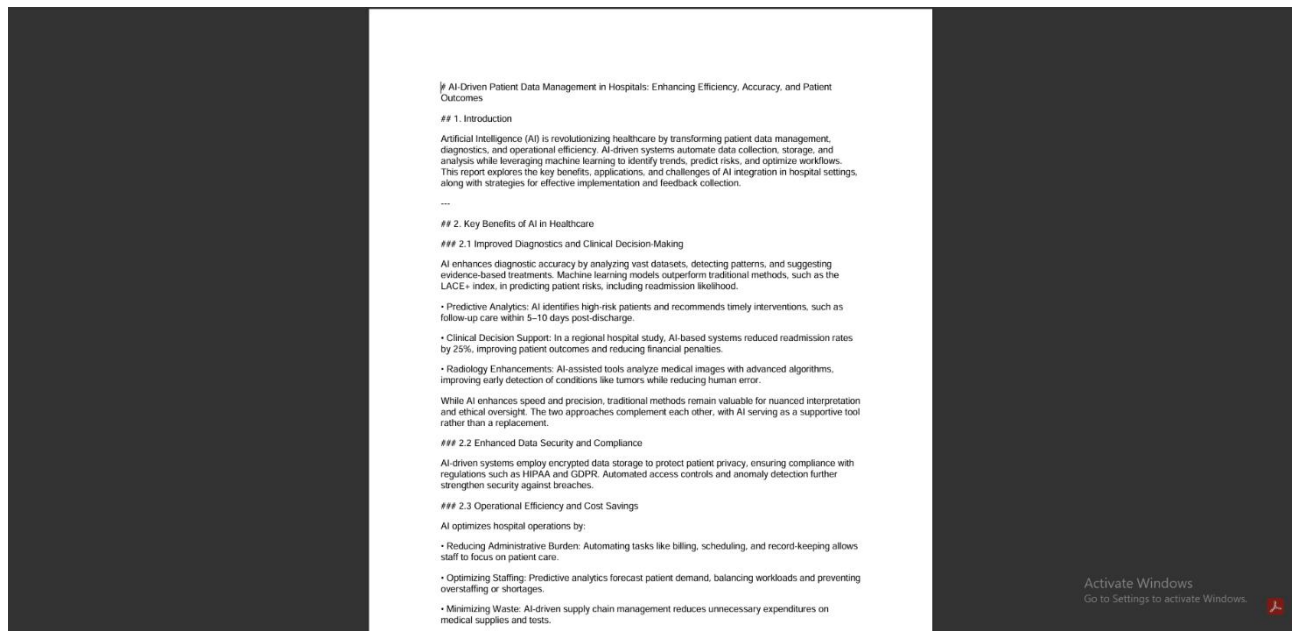


Figure : Generated TXT Report

```
File Edit View H1 ~ 100 B 7 5 0 田 ~ 4

# **Report: AI Implementation in Hospital Emergency Departments - Efficiency, Challenges, and Ethical Considerations**

## **1. Introduction**
Artificial Intelligence (AI) is transforming healthcare delivery, particularly in hospital emergency departments (EDs), by enhancing efficiency, diagnostic accuracy, and patient outcomes. This report examines the benefits, implementation strategies, financial implications, and ethical considerations of AI adoption in emergency care settings.
---

## **2. Benefits of AI in Emergency Departments**

### **2.1 Improved Efficiency and Workflow Optimization**
AI-driven tools streamline emergency care by:
- **Prioritizing critical cases** (e.g., **Aidoc** flags strokes in CT scans, reducing response times).
- **Reducing human error** (e.g., **IBM Watson Health** assists in sepsis diagnosis, lowering mortality rates).
- **Automating administrative tasks**, freeing staff for patient care.

Key advantages include:
- **Faster decision-making** through real-time data analysis.
- **Optimized resource allocation**, reducing bottlenecks.

### **2.2 Enhanced Patient Outcomes**
AI integration improves clinical metrics by:
- **Increasing diagnostic accuracy** (e.g., detecting subtle patterns in imaging or lab results).
- **Shortening diagnosis-to-treatment cycles** via automated analysis.
- **Reducing misdiagnoses and human oversights**, leading to better recovery rates.
---

## **3. Implementation Strategies**

### **3.1 Phased Deployment Across Departments**
Hospitals adopt AI in stages to ensure smooth integration:

| **Phase** | **Duration** | **Key Applications** |
|-----|-----|-----|
| **Early Phase** | 0-2 years | Radiology (image analysis), pathology (diagnostics), administrative (scheduling, billing). |
| **Mid Phase** | 2-5 years | Emergency care (real-time monitoring), ICU (predictive analytics), pharmacy (drug interaction checks). |
| **Long-term** | 5+ years | Surgery (robotic assistance), mental health (AI chatbots), personalized treatment planning. |

### **3.2 Workflow Efficiency Assessment**
To evaluate AI's impact, hospitals should:
- **Conduct structured staff interviews** to assess:
  - **Efficiency gains** (e.g., time saved, reduced administrative tasks).
  - **Challenges** (e.g., training needs, system integration issues).
  - **Patient outcomes** (e.g., improved care, workflow bottlenecks).
- **Compare responses across departments** to identify patterns.

**Sample Interview Questions:**
- Which AI tools are used in your department, and how have they impacted daily tasks?
- What challenges have you encountered with AI integration?
- Have you observed improvements in patient care or operational efficiency?
```

6.4 Chapter Summary

This chapter described the implementation of the Cognix system using Python and frameworks such as LangGraph, LangChain, Streamlit, and Mistral AI. It explained how each agent was developed and integrated into the workflow, along with the use of LangSmith for monitoring and ReportLab for PDF generation. The chapter demonstrated how the system was successfully built and executed.

CHAPTER 7 : RESULTS AND OUTPUT ANALYSIS

7.1 Introduction

The Results and Output Analysis chapter presents the outputs generated by the Autonomous Cognitive Machine (Cognix). The system was developed to autonomously analyze user goals, create execution plans, perform research, generate solutions, validate results, and produce final reports.

The results demonstrate the effectiveness of the multi-agent architecture and show how different agents collaborate to achieve the desired objective.

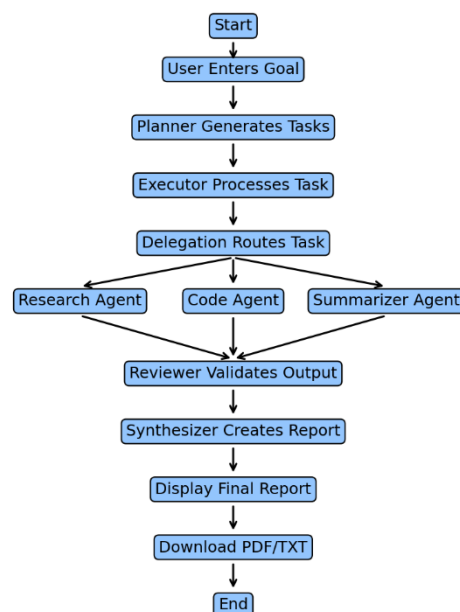
7.2 Workflow Execution

The Cognix system follows a structured workflow where multiple agents work together to process a user-defined goal.

Workflow Steps

- User submits a goal.
 - Planner Agent analyzes the goal.
 - Task list is generated.
 - Executor Agent assigns tasks.
 - Specialized agents perform execution.
 - Reviewer Agent validates outputs.
 - LangSmith traces the workflow.
 - Synthesizer Agent generates the final response.
 - PDF report is created.
-

7.3 Complete Workflow Execution



7.4 Goal Processing Result

The first stage of execution begins when the user enters a goal through the Streamlit interface.

The system successfully accepts the objective and initiates the cogniti

7.5 User Goal Submission Screen



Result Analysis

- User input accepted successfully.
- Workflow execution initiated.
- Goal transferred to Planner Agent.

7.6 Planner Agent Output

The Planner Agent analyzes the user goal and generates a structured execution plan.

Functions Performed

- Goal understanding
- Task decomposition
- Planning

Result Analysis

The Planner Agent successfully generated a detailed list of tasks required to achieve the user objective. The generated plan improved workflow organization and execution efficiency.

7.7 Research Agent Output

The Research Agent retrieves relevant information using external search capabilities.

Functions Performed

- Information retrieval
- Data collection
- Knowledge gathering

Result Analysis

The agent successfully collected relevant information and provided useful insights required for task completion.

7.8Code Agent Output

The Code Agent generates programming solutions and technical content whenever required.

Functions Performed

- Code generation
- Technical assistance
- Programming support

Result Analysis

The generated code was relevant to the user requirements and contributed effectively to task completion.

7.9 Summarizer Agent Output

The Summarizer Agent converts lengthy information into concise and understandable content.

Functions Performed

- Content summarization
- Information refinement
- Output optimization

Result Analysis

The generated code was relevant to the user requirements and contributed effectively to task completion.

7.10 Reviewer Agent Output

The Reviewer Agent evaluates generated outputs and verifies their quality before final synthesis.

Functions Performed

- Output validation
- Quality checking
- Error identification

Result Analysis

The Reviewer Agent successfully verified output quality and ensured consistency before report generation.

7.11 LangSmith Workflow Analysis

LangSmith was integrated into the Cognix system to monitor workflow execution and agent interactions.

Features Observed

- Agent execution tracing
- Prompt tracking
- Response monitoring
- Workflow visualization
- Performance analysis

[INSERT SCREENSHOT 8.7 HERE – LangSmith Dashboard]

[INSERT SCREENSHOT 8.8 HERE – LangSmith Trace Visualization]

Result Analysis

LangSmith provided complete visibility into the execution flow of the system. It helped monitor agent behavior and analyze workflow performance effectively.

7.12 Synthesizer Agent Output

The Synthesizer Agent combines validated outputs from all agents into a structured final response.

Functions Performed

- Output aggregation
- Report preparation
- Final response generation

Result Analysis

The generated response successfully combined outputs from multiple agents and presented them in a coherent

format.

7.13 Generated PDF Report

After successful workflow completion, the final report is automatically converted into PDF format.

Features

- Structured document format
- Automated generation
- Easy sharing and storage

Result Analysis

The PDF report was generated successfully and contained all outputs produced during execution.

7.14 Overall Performance Analysis

The Cognix system demonstrated effective collaboration among multiple autonomous agents.

Observations

- Goals were processed successfully.
- Tasks were generated accurately.
- Agents collaborated efficiently.
- Outputs were validated before finalization.
- Workflow monitoring was achieved through LangSmith.
- PDF reports were generated automatically.

Advantages Observed

- Reduced manual effort.
- Improved workflow organization.
- Better output quality.
- Enhanced monitoring and debugging.
- Scalable multi-agent architecture.

7.15 Chapter Summary

This chapter presented the outputs generated by the Autonomous Cognitive Machine (Cognix) and analyzed the performance of each agent involved in the workflow. The results demonstrate that the system successfully performs autonomous planning, execution, validation, synthesis, and report generation. The integration of LangSmith further enhanced workflow visibility and monitoring, making the system more reliable and efficient.

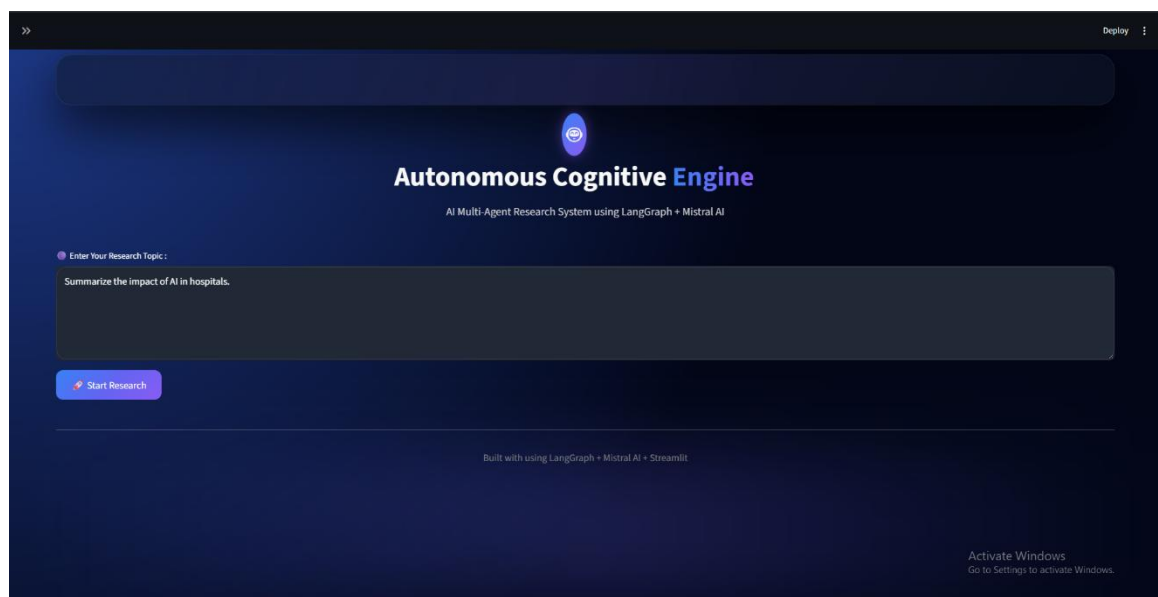
CHAPTER 8 : SCREENSHOTS

8.1 Introduction

This chapter presents the screenshots of the Autonomous Cognitive Machine (Cognix) developed during the internship. The screenshots illustrate the user interface, workflow execution, agent interactions, LangSmith monitoring, and report generation process. These outputs demonstrate the practical implementation and functionality of the system.

8.2 Streamlit Home Page

The Streamlit Home Page serves as the main interface of the Cognix system. It allows users to enter their goals and initiate the workflow execution process.



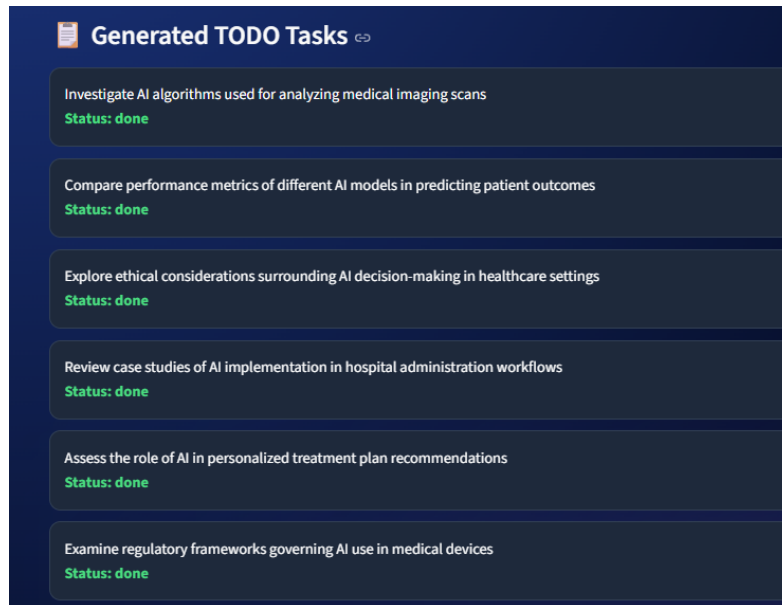
8.3 User Goal Submission

The user enters a task or objective through the input field provided in the Streamlit interface.



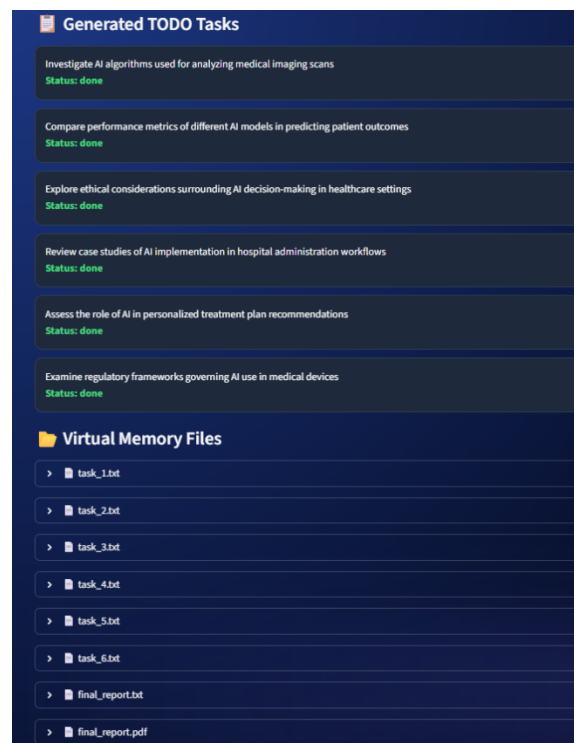
8.4 Planner Agent Output

The Planner Agent analyzes the user objective and generates a structured execution plan.



8.5 Executor Agent Output

The Executor Agent manages the execution of tasks and coordinates communication among specialized agents.



8.6 Final Report

The Final Report is combines validated outputs from all agents into a structured final response.

Final Generated Report

AI-Driven Patient Data Management in Hospitals: Enhancing Efficiency, Accuracy, and Patient Outcomes

1. Introduction

Artificial Intelligence (AI) is revolutionizing healthcare by transforming patient data management, diagnostics, and operational efficiency. AI-driven systems automate data collection, storage, and analysis while leveraging machine learning to identify trends, predict risks, and optimize workflows. This report explores the key benefits, applications, and challenges of AI integration in hospital settings, along with strategies for effective implementation and feedback collection.

2. Key Benefits of AI in Healthcare

2.1 Improved Diagnostics and Clinical Decision-Making

AI enhances diagnostic accuracy by analyzing vast datasets, detecting patterns, and suggesting evidence-based treatments. Machine learning models outperform traditional methods, such as the LACE+ index, in predicting patient risks, including readmission likelihood.

Predictive Analytics: AI identifies high-risk patients and recommends timely interventions, such as follow-up care within 5–10 days post-discharge.

Clinical Decision Support: In a regional hospital study, AI-based systems reduced readmission rates by 25%, improving patient outcomes and reducing financial penalties.

Radiology Enhancements: AI-assisted tools analyze medical images with advanced algorithms, improving early detection of conditions like tumors while reducing human error.

While AI enhances speed and precision, traditional methods remain valuable for nuanced interpretation and ethical oversight. The two approaches complement each other, with AI serving as a supportive tool rather than a replacement.

8.7 Generate PDF Report

The final output generated by the Cognix system is automatically converted into PDF format.

AI-Driven Patient Data Management in Hospitals: Enhancing Efficiency, Accuracy, and Patient Outcomes

1. Introduction

Artificial Intelligence (AI) is revolutionizing healthcare by transforming patient data management, diagnostics, and operational efficiency. AI-driven systems automate data collection, storage, and analysis while leveraging machine learning to identify trends, predict risks, and optimize workflows. This report explores the key benefits, applications, and challenges of AI integration in hospital settings, along with strategies for effective implementation and feedback collection.

...

2. Key Benefits of AI in Healthcare

2.1 Improved Diagnostics and Clinical Decision-Making

AI enhances diagnostic accuracy by analyzing vast datasets, detecting patterns, and suggesting evidence-based treatments. Machine learning models outperform traditional methods, such as the LACE+ index, in predicting patient risks, including readmission likelihood.

• Predictive Analytics: AI identifies high-risk patients and recommends timely interventions, such as follow-up care within 5–10 days post-discharge.

• Clinical Decision Support: In a regional hospital study, AI-based systems reduced readmission rates by 25%, improving patient outcomes and reducing financial penalties.

• Radiology Enhancements: AI-assisted tools analyze medical images with advanced algorithms, improving early detection of conditions like tumors while reducing human error.

While AI enhances speed and precision, traditional methods remain valuable for nuanced interpretation and ethical oversight. The two approaches complement each other, with AI serving as a supportive tool rather than a replacement.

2.2 Enhanced Data Security and Compliance

AI-driven systems employ encrypted data storage to protect patient privacy, ensuring compliance with regulations such as HIPAA and GDPR. Automated access controls and anomaly detection further strengthen security against breaches.

2.3 Operational Efficiency and Cost Savings

AI optimizes hospital operations by:

• Reducing Administrative Burden: Automating tasks like billing, scheduling, and record-keeping allows staff to focus on patient care.

• Optimizing Staffing: Predictive analytics forecast patient demand, balancing workloads and preventing overstaffing or shortages.

• Minimizing Waste: AI-driven supply chain management reduces unnecessary expenditures on medical supplies and tests.

Activate Windows

Go to Settings to activate Windows.

8.8 Langsmith Dashboard Output

8.9 Chapter Summary

This chapter presented the screenshots and output screens of the Autonomous Cognitive Machine (Cognix). The screenshots demonstrated the implementation of the user interface, agent execution process, LangGraph workflow, LangSmith monitoring, and PDF report generation. These outputs validate the successful functioning of the developed multi

CHAPTER 9 : CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Autonomous Cognitive Machine (Cognix) was successfully designed and developed as a multi-agent cognitive system capable of autonomously performing planning, execution, validation, and report generation. The project demonstrates how multiple intelligent agents can collaborate to solve complex tasks efficiently and produce meaningful results with minimal human intervention.

The system utilizes modern AI technologies such as LangGraph, LangChain, Mistral AI, Streamlit, Tavily Search API, and LangSmith to create a scalable and intelligent workflow. The Planner Agent decomposes user goals into manageable tasks, the Executor Agent coordinates task execution, the Research and Code Agents provide specialized capabilities, the Reviewer Agent ensures output quality, and the Synthesizer Agent generates the final structured response.

The integration of LangSmith enhanced the observability of the system by enabling workflow tracing, performance monitoring, and debugging. Additionally, the automatic PDF report generation feature improved documentation and result presentation.

Overall, the project successfully achieved its objectives and demonstrated the practical implementation of a cognitive multi-agent architecture for intelligent task automation.

9.2 Achievements

The major achievements of the project are:

- Developed a fully functional autonomous cognitive multi-agent system.
- Implemented task planning and decomposition using AI.
- Enabled collaboration among multiple specialized agents.
- Integrated external knowledge retrieval capabilities.
- Implemented automated output validation.
- Integrated LangSmith for tracing and monitoring.
- Generated structured PDF reports automatically.
- Developed an interactive Streamlit-based user interface.
- Created a scalable and modular architecture.

9.3 Limitations

Although the system performs effectively, certain limitations exist:

- Performance depends on the quality of AI model responses.
- External API availability may affect execution.
- Complex workflows may increase execution time.
- The system currently supports a limited set of specialized agents.
- Internet connectivity is required for certain functionalities.

These limitations can be addressed through future enhancements and optimization.

9.4 Future Scope

The Autonomous Cognitive Machine can be further enhanced by incorporating additional advanced features.

Future Enhancements

- Integration of more specialized AI agents.
- Support for multimodal inputs such as images, audio, and video.
- Advanced memory management for long-term task retention.
- Real-time collaboration between multiple users.
- Deployment on cloud platforms for large-scale usage.
- Integration with enterprise applications and databases.
- Enhanced analytics and performance dashboards.
- Voice-based interaction and conversational interfaces.
- Improved autonomous decision-making capabilities.
- Support for additional Large Language Models.

These enhancements can significantly improve the capabilities and applicability of the system in real-world environments.

9.5 Final Remarks

The Autonomous Cognitive Machine (Cognix) represents an important step toward the development of intelligent autonomous systems capable of performing complex tasks through collaborative agent-based architectures. The project successfully combines cognitive computing, multi-agent systems, workflow orchestration, and generative AI technologies to create a practical and scalable solution.

The experience gained during the internship and project development contributed significantly to understanding modern AI technologies, software engineering practices, and intelligent system design

9.6 Chapter Summary

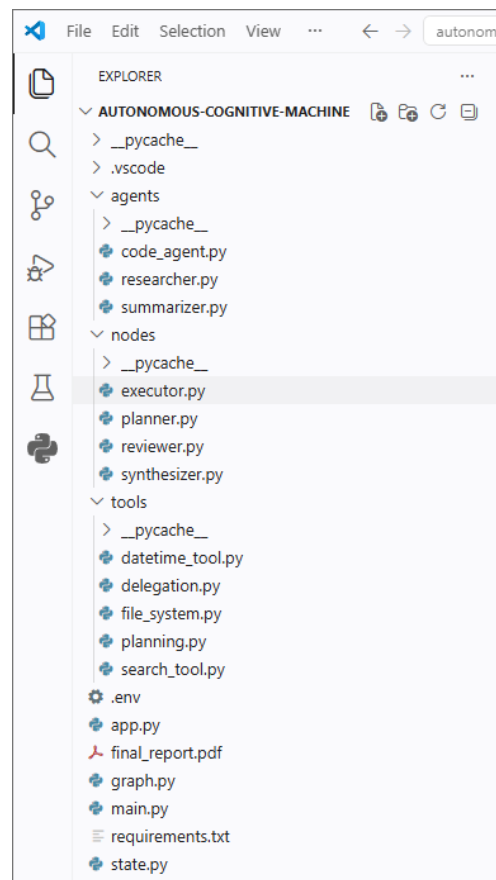
This chapter summarized the overall project, highlighted its achievements, discussed current limitations, and presented possible future enhancements. The successful implementation of the Cognix system demonstrates the effectiveness of autonomous cognitive multi-agent architectures in solving complex real-world problems and generating intelligent outcomes.

REFERENCES

- LangGraph Documentation. Available at: <https://langchain-ai.github.io/langgraph/>
- LangChain Documentation. Available at: <https://python.langchain.com/>
- LangSmith Documentation. Available at: <https://docs.smith.langchain.com/>
- Mistral AI Documentation. Available at: <https://docs.mistral.ai/>
- Streamlit Documentation. Available at: <https://docs.streamlit.io/>
- Tavily Search API Documentation. Available at: <https://docs.tavily.com/>
- Python Software Foundation. Python Documentation. Available at: <https://docs.python.org/3/>

APPENDIX

A.1 Project Folder Structure



The Cognix project follows a modular architecture where each agent and utility component is organized into separate files and directories to improve maintainability and scalability.

A.2 Environment Configuration

The project uses environment variables to securely manage API keys and configuration settings.

Example:

```
MISTRAL_API_KEY = "your_api_key"
TAVILY_API_KEY = "your_api_key"
LANGCHAIN_API_KEY="your_api_key"
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=Project_name
```

A.3 Sample Code Snippet – Planner Agent

```
def planner_node(state):
    goal = state["messages"][-1]
    raw_todos = write_todos(goal)
    structured_todos = []

    for item in raw_todos:
        structured_todos.append({
            "title": item.get("title", "Untitled Task"),
            "description": item.get("description", "No description"),
            "status": "pending"
        })
    state["todos"] = structured_todos
    return state
```

A.4 Sample Code Snippet – LangGraph Workflow

```
def build_graph():
    graph = StateGraph(AgentState)

    graph.add_node("planner", planner_node)
    graph.add_node("executor", executor_node)
    graph.add_node("reviewer", reviewer_node)
    graph.add_node("synthesizer", synthesizer_node)
```

A.5 Sample Code Snippet – LangSmith Integration

```
LANGCHAIN_API_KEY="your_api_key"
LANGCHAIN_TRACING_V2=true
LANGCHAIN_PROJECT=Cognix
```

A.6 Sample Code Snippet – PDF Generation

```
def create_pdf(text, filename="final_report.pdf"):
    doc = SimpleDocTemplate(filename, pagesize=letter)
    styles = getSampleStyleSheet()
```

A.7 Internship Learning Outcomes

During the internship at Infosys, the following skills were acquired:

- Multi-Agent System Development
- LangGraph Workflow Design
- LangChain Integration
- LangSmith Monitoring and Tracing
- Prompt Engineering
- Streamlit Application Development
- API Integration
- PDF and TXT Report Generation
- AI Workflow Orchestration
- Software Development Best Practices